A Robust Mechanism for Congestion Control: INC*

Elvira Baydal and Pedro López

Dept. of Computer Engineering (DISCA)
Universidad Politécnica de Valencia
Camino de Vera s/n, 46071 – Valencia, SPAIN
elvira, plopez@disca.upv.es

Abstract. Several techniques to prevent congestion in multiprocessor interconnection networks have been recently proposed. Unfortunately, they either suffer from a lack of robustness or detect congestion relying on global information that wastes a lot of transmission resources.

This paper presents a new mechanism that uses only local information to avoid network saturation in wormhole networks. It is robust and works properly in different conditions. It first applies preventive measures of different intensity depending on the estimated traffic level; and if necessary, it uses message throttling during predefined time intervals that are extended if congestion is repeatedly detected. Evaluation results for different network loads and topologies show that the proposed mechanism avoids network performance degradation. Most important, without introducing any penalty for low and medium network loads.

1 Introduction

Massively parallel computers provide the performance that scientific and commercial applications require. Their interconnection networks offer the low latencies and high bandwidth that is needed for different kinds of traffic. Usually, wormhole switching with virtual channels and adaptive routing is used. However, multiprocessor interconnection networks may suffer from severe saturation problems with high traffic loads, which may prevent reaching the wished performance (see Figure 1). With low and medium network loads, the accepted traffic rate is the same as the injection rate. But if traffic increases and reaches (or surpasses) certain level (the saturation point), accepted traffic falls and message latency increases considerably. Performance degradation appears because with high network traffic several packets compete for the same resources (physical or virtual channels), but as only one packet can use them, the rest of the packets stay stopped in the network, thus blocking other packets and so on. When this situation is generalized, the network is congested and performance degradation appears.

One of the most popular approaches to solve this problem is to avoid that network traffic reaches the saturation point. Message throttling [4] (stop or slow down the injection rate of messages) has been the most frequently used method. Several mechanisms for multiprocessors have already been proposed but they have important drawbacks [2].

* This work was supported by by the Spanish MCYT under Grants TIC2000-1151-C07 and 1FD97-2129

H. Kosch, L. Böszörményi, H. Hellwagner (Eds.): Euro-Par 2003, LNCS 2790, pp. 958–968, 2003.
© Springer-Verlag Berlin Heidelberg 2003

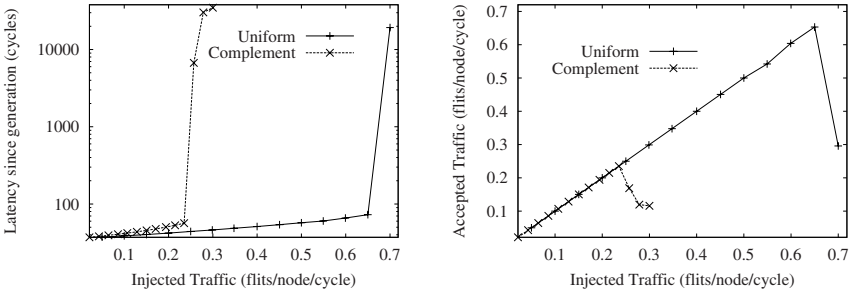


Fig. 1. Performance degradation in a 3-cube 8-ary (512 nodes) with a deadlock recovery-based fully adaptive routing algorithm with 3 virtual channels per physical channel. Uniform and complement traffic patterns. 16-flit messages.

In this paper, we propose and evaluate a new mechanism to avoid network saturation that tries to overcome these drawbacks. It is a robust mechanism that applies preventive measures of different intensity depending on the network traffic. Moreover, it progressively reduces the bandwidth available to inject messages, either by reducing the number of injection channels or, if it is necessary, by applying increasing intervals of forbidden message injection. The mechanism has been exhaustively tested, achieving good results in all the evaluated conditions and improving recent proposals [2], [15].

The rest of the paper is organized as follows. In order to design a good congestion control mechanism, Section 2 presents the main desirable features of such a mechanism. Section 3 describes the new proposal. Section 4 presents some simulation results. Finally, some conclusions are obtained.

2 Motivation

First, the mechanism should work properly for different conditions (robustness): different message destination distributions, message sizes, network topologies and even routing algorithms. As we can see in Figure 1, the network saturation point depends on the traffic pattern. Hence, a suitable mechanism for one distribution, may not work properly with another one. The same problem may appear when we change network topology. However, many of the previously proposed mechanisms have been analyzed with only one network size [1], [15] and for the uniform distribution of message destinations [4], [8], [13]. Other mechanisms do not achieve good results all traffic patterns considered [14]. On the other hand, some mechanisms are based on the idea that all the virtual channels of a physical channel are used in the same way [1], [2], thus being well suited for deadlock recovery schemes but may not for deadlock avoidance ones.

Second, the mechanism should not penalize the network when it is not saturated. Notice that this is the most frequent situation [12]. Thus, when with low and medium network traffic, the mechanism should not restrict or delay message injection into the network. However, some of the previous proposals increase message latency before the saturation point [4], [14], also reducing the network throughput.

Finally, the new mechanism should not generate new problems in the network. Those proposals that need to send extra information across the network increase network traffic and may make worse the congestion situation [8]. Moreover, when congestion detection relies on global information, mechanisms do not scale well since the amount of network status information increases with the network size [15].

3 The New Mechanism: Injection and Network Congestion

As we stated in Section 1, when network traffic is near to the saturation point, many packets block. Indeed, as wormhole switching is used, these packets stay stopped in the network, thus blocking other packets. In addition, as physical channels are multiplexed into more virtual channels, message advance speed decreases.

The new mechanism uses message advance speed in order to detect network congestion. In particular, it measures the number of transmitted flits during a fixed time interval t_m. Each virtual channel has an associated counter that increases each time one flit is transmitted. At the end of each interval, the channel is considered congested if it is busy and the number of transmitted flits is lower than a threshold *fc* (*flit counter*). When the header of a message reaches a congested area, either it will block or advance at a low speed. As a consequence, the remaining flits of the message will also block or decrease their advance speed. By tracking this speed, we try to detect congestion in network areas remote to the current node, but using only local information.

Once congestion is detected, the applied measures are different depending on whether the node is currently injecting messages towards the congested area. In particular, each node manage two different flags: the *injection* and the *network* flags. The *injection* flag is set when some of the virtual channels that the node is using to inject messages into the network are congested. On the contrary, the *network* flag is set when congestion is detected in any virtual channel that is being used by messages injected by other nodes. The mechanism will referred to as INC, *Injection and Network Congestion* detection.

When either the injection or network flags are set, message injection restrictions are applied. These measures are more restrictive when the injection flag is set than when only network flag does. The injection flag setting means that the node is actively contributing to network congestion. On the contrary, when only the network flag is set, although the current node is not directly generating the problem, as congestion propagates quickly through the network, some preventive measures should be applied. In both cases, if congestion is repeatedly detected in later intervals, the measures get more restrictive. Thus, the mechanism works in a progressive way.

In particular, when injection congestion is detected, the number of enabled injection channels is divided (integer division) by a factor r_1. If this quotient reaches zero, injection is completely stopped but only during some interval f_{ii} (*forbidden injection interval*). This interval starts when the last message that the node is currently injecting is completely injected into the network. After f_{ii} cycles, one injection channel is enabled regardless of the detected network traffic, and the first pending message, if any, is injected. As network traffic is estimated periodically, injection restrictions get harder or, on the contrary, are reduced. The mechanism uses limited forbidden injection intervals for two reasons. First, it avoids starvation and, second, it allows that other nodes also detect congestion and

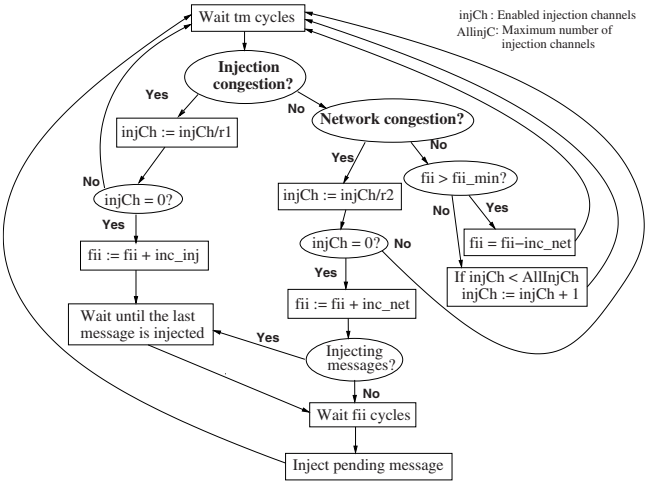


Fig. 2. Operation of the congestion control mechanism INC.

apply message injection limitation. If only network congestion is detected, the enabled injection channels are reduced by r_2, with $r_2 < r_1$. Forbidden injection intervals are also used if network congestion is repeatedly detected, but they increase more slowly.

The values of r_1 and r_2 depend on the network radix k. Notice that when k increases also does the average distance traversed by the messages, which makes worse network congestion. Therefore injection restrictions have to be harder. After some tests, we have found that $r_1 = k/4$ and $r_2 = k/8$ work well, k being the network radix.

The forbidden injection interval is initialized to some value $f_{ii_{min}}$. Every time congestion is detected, it is incremented. This increment again depends on the network radix k. We have chosen it higher for injection than for network congestion. After several tests, we have found that values $inc_{inj} = (f_{ii_{min}} \times k)/16$ and $inc_{net} = (f_{ii_{min}} \times k)/32$ work well, for injection and network congestion, respectively.

Finally, injection bandwidth has to be recovered when congestion it not longer detected. However, injection restrictions are also reduced gradually. After an interval without detecting any congestion, injection limitation is smoothed. First, injection bandwidth is increased by reducing the forbidden interval (by inc_{net}), and later (when it reaches the minimum value $f_{ii_{min}}$), by increasing (by one channel at a time) the number of injection channels. Notice that the reduction of injection bandwidth is faster than the later recovering. Figure 2 summarizes the behavior of the mechanism.

4 Evaluation

In this section, we will evaluate by simulation the behavior of the INC mechanism. The evaluation methodology is based on the one proposed in [2]. The most important performance measures are latency (time required to deliver a message, including the time spent at the source queue, measured in clock cycles) and throughput (maximum traffic

accepted by the network, measured in flits per node per cycle). On the other hand, as message latency at saturation grows with time, another valuable measure of performance is the required time to deliver a given number of messages.

4.1 Network Model

Each node has a router, a crossbar switch and several physical channels. Routing time and both transmission time across the crossbar and across a channel are all assumed to be equal to one clock cycle. Each node has four injection/ejection channels. We explained the advantages of having several injection/ejection channels in [2].

Concerning deadlock handling, we have evaluated the new mechanism both with deadlock avoidance and recovery techniques. For deadlock recovery, we use software-based deadlock recovery [11] and a True Fully Adaptive routing algorithm (TFAR) [12,11]. This routing algorithm allows the use of any virtual channel of those physical channels that forwards a message closer to its destination. Deadlocks are detected with the mechanism proposed in [10] with a threshold equal to 32 cycles. With deadlock avoidance, we use the fully adaptive routing algorithm proposed in [6]. In both cases, we have tested the mechanism with 3 and 4 virtual channels per physical channel.

We have evaluated the performance of the proposed congestion control mechanism on different bidirectional k-ary n-cubes. In particular, we have used the following network sizes: 256 nodes (n=2, k=16), 1024 nodes (n=2, k=32), 512 nodes (n=3, k=8), and 4096 nodes (n=3, k=16).

4.2 Network Load

Each node generates messages independently, according to an exponential distribution. Destinations are chosen according to the *Uniform, Uniform with locality, Butterfly, Complement, Bit-reversal,* and *Perfect-shuffle* traffic patterns. Uniform distribution is the most frequently used one in the analysis of interconnection networks. The other patterns take into account the permutations that are usually performed in parallel numerical algorithms [9]. For message length, 16-flit, 64-flit and 128-flit messages are considered.

We have performed two kinds of experiments, using constant and dynamic loads. Experiments with constant loads are the usual evaluation tool for interconnection networks. In this case, the message generation rate is constant and the same for all network nodes. We analyze all the traffic range, from low load until saturation. Simulations finish after receiving 500,000 messages with the smallest networks (256 and 512 nodes) and 1,200,000 with the largest ones (1024 and 4096 nodes), but only the last 300,000 (1,000,000) messages are considered to calculate average latencies.

We will also evaluate the mechanism behavior when the traffic conditions change dynamically. In particular, we use a bursty load that alternates periods of a high message injection rate L_{sat} (that corresponds to a saturation interval), with periods of low traffic L_{low}. These loads are repeated twice ($L_{sat}L_{low}L_{sat}L_{low}$). Let L_s be the message generation rate just before the network reaches saturation. We have chosen a generation rate $L_{low} = 0.5 \times L_s$. On the other hand, in order to analyze the mechanism behavior at different saturation levels, we have tested two different values of the saturation load: $L_{sat} = 1.2 \times L_s$ and $L_{sat} = 1.8 \times L_s$. Concerning the number of sent messages, for

a given generation rate, each node generates the same number of messages. When a node has generated all the messages for one of the injection rates, then, it starts generating traffic with the next load level in sequence. Finally, when it has generated all the messages for the complete sequence, it does not generate new messages at all. The simulation finishes when all the injected messages arrive to their destinations.

We show results with $1,000$ messages per node generated with the L_{sat} injection rate, and two different values for the load under the saturation (L_{low}): $M_{low} = 555$ and $M_{low} = 1111$ messages. The number of messages generated with the L_{low} rate establishes the elapsed time between two traffic bursts, and it allows us to analyze how the mechanism recovers the network after a congestion period.

4.3 Performance Comparison

In this section, we will analyze the behavior of the **INC** mechanism proposed in this paper. For comparison purposes, we will also evaluate the **U-channels** ([2]) and **Self-Tuned** ([15]) mechanisms. **U-channels** also uses only local information to avoid network saturation. Each node locally estimates traffic by using the percentage of free virtual output channels that can be used to forward a message towards its destination. When this number is below a threshold value, congestion network is assumed to exist and message throttling is applied. In the **Self-Tuned** mechanism, nodes detect network congestion by using global information about the number of full buffers in the network. If this number surpasses a threshold, all nodes apply message throttling. The use of global information requires to broadcast data among all the network nodes. A way of transmitting this control information is to use a sideband network [15]. As the mechanism proposed in this paper does not need to exchange control messages, to make a fair comparison, the bandwidth provided by the sideband network should be considered as additional available bandwidth in the main interconnection network. However, in the results that we present we do not consider this fact. If this additional bandwidth were considered, the differences, not only in throughput but also in latency, between **Self-Tuned** and the rest of the mechanisms would be greater than the ones shown.

As it was shown in [2], **U-channels** outperforms **Self-Tuned** with deadlock recovery-based TFAR routing, but it is not well suited for deadlock avoidance since, in this case, all the virtual channels are not used in the same way. For this reason, we compare **INC** with **U-channels** when TFAR routing is used, while we use **Self-Tuned** as the comparison with deadlock avoidance. Finally, in all cases, results without any congestion control mechanism (**No-Lim**) are also shown.

First of all, the **INC** mechanism has to be tuned. We have evaluated its behavior with different message destination distributions, message sizes and network topologies. In all cases, the number of transmitted flits per virtual channel is sampled every $t_m = 20$ cycles. As an example, Figure 3 shows the average message latency versus traffic for different threshold values for the uniform and perfect-shuffle traffic patterns with the TFAR routing algorithm based on deadlock recovery. Notice that both latency and accepted traffic are dependent variables on offered traffic. As you can see, threshold adjustment is not critical. In fact, several of them work fine since the mechanism tries to find the optimal injection bandwidth by applying progressive measures. Table 1 shows the values chosen for the evaluated networks, different number of virtual channels per physical channel,

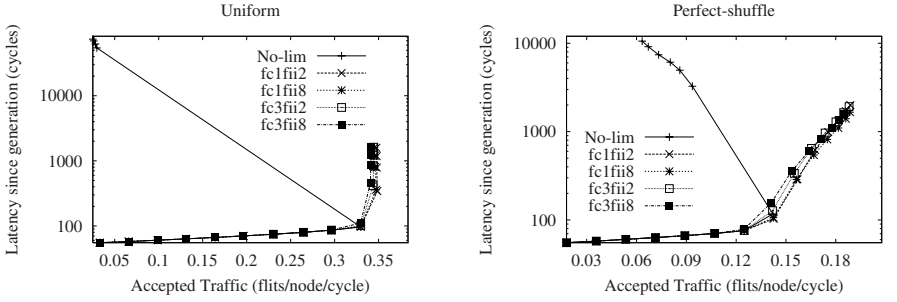


Fig. 3. Average message latency vs. accepted traffic for different threshold values for the *INC* mechanism. 16-ary 3-cube (4096 nodes). 3 virtual channels per physical channel. TFAR algorithm based on deadlock recovery.

Table 1. Threshold values for different topologies and number of virtual channels per physical channel.

Nodes	Virt. ch.	Deadlock recovery				Deadlock avoidance			
		fc	$f_{ii_{min}}$	inc_{inj}	inc_{net}	fc	$f_{ii_{min}}$	inc_{inj}	inc_{net}
256	3	1	4	4	2	1	4	4	2
	4	1	2	2	1	1	2	2	1
1024	3	4	2	4	1	6	8	16	8
	4	2	1	2	1	5	2	4	2
512	3	1	4	2	1	1	1	0	0
	4	1	1	0	0	1	1	0	0
4096	3	1	2	2	1	2	2	2	1
	4	1	2	2	1	1	2	2	1

and deadlock recovery and avoidance strategies. As you can see, the optimal values depend mainly on the topology radix k and the number of virtual channels per physical channel. This is not a problem, as these are design parameters that are fixed once the machine is built. Notice that for 512 nodes and 4 virtual channels, the f_{ii} is held constant ($inc_{inj} = inc_{net} = 0$). In this case, only reducing the number of injection channels is enough to avoid congestion.

Once tuned the new mechanism, we can compare it with the other ones. For the sake of shortness, we will only show some results for the 4096-node network (k=16, n=3) and 3 virtual channels per physical channel. The complete evaluation for all the tested networks, traffic patterns and different number of virtual channels per physical channel can be found in [3]. Figure 4 shows these results.

As we can see, the new mechanism **INC** avoids the performance degradation in all the evaluated cases, without increasing latencies for low and medium loads. In general, the results obtained with deadlock recovery are quite similar to the **U-channels** ones. On the other hand, with deadlock avoidance the **INC** mechanism improves in all the cases the **Self-tuned**. Although the **Self-Tuned** mechanism helps in alleviating network

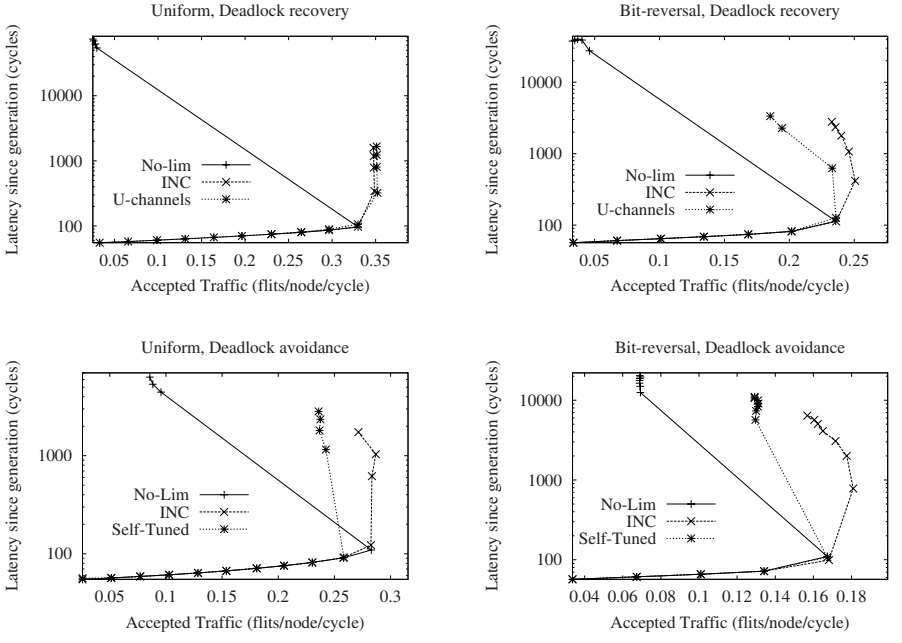


Fig. 4. Average message latency vs. accepted traffic. 16-flit messages. 8-ary 3-cube (4096 nodes).

congestion, it reduces network throughput and increases network latency with medium loads in some cases.

Figure 5 shows some results for dynamic loads with a uniform distribution of message destinations, $L_{sat} = 1.8 \times L_s$ and $M_{low} = 1111$ messages per node with L_{low}. As we can see, with the **INC** mechanism, the network accepts the injected bursty traffic without problems. On the contrary, when no congestion control mechanism is applied, as soon as the first burst is applied into the network, congestion appears. As a consequence, latency strongly increases and accepted traffic falls down. Later, after some time injecting with low rate, network traffic starts recovering but the arrival of a new traffic burst prevents it. Concerning **Self-Tuned** mechanism, we can see that it excessively limits injection rate, significantly reducing the highest value of accepted traffic, therefore increasing the time required to deliver all the injected messages. As we stated before, this time is another performance measure that is strongly affected by network saturation. Figure 5 shows that **INC** delivers the required number of messages in nearly half the time than **No-Lim** (74.36% of improvement), while **Self-Tuned** achieves a slightly smaller improvement (70.51%).

Table 2 shows the percentage of improvement in the time required to deliver all the injected messages, for different network sizes, when we apply the **INC** and **U-channels** congestion control mechanisms versus when no congestion control mechanism is used. Deadlock recovery is used. We have got improvements in the simulation time up to 156%

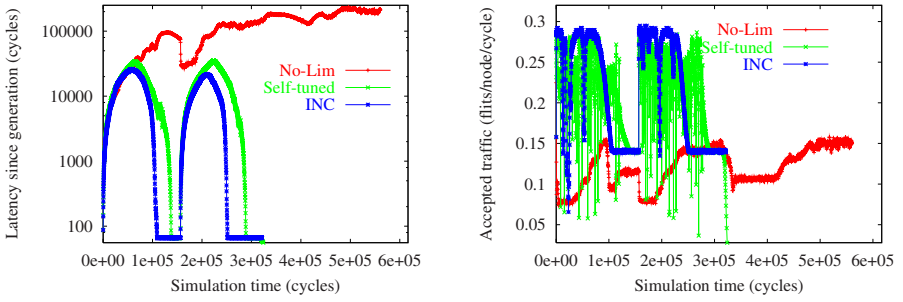


Fig. 5. Performance with dynamic loads. Uniform traffic pattern. 16-flit messages. 16-ary 3-cube (4096 nodes), 3 virtual channels. Fully adaptive algorithm with deadlock avoidance.

Table 2. Improvement (%) in the time required to deliver all the injected messages for the *INC* and *U-channels* mechanisms with different topologies, *TFAR* algorithm with deadlock recovery and 3 virtual channels. Uniform traffic pattern.

Nodes	Mechanism	$L_{sat} = 1.2 \times T_s$		$L_{sat} = 1.8 \times T_s$	
		$M_{low} = 1111$	$M_{low} = 555$	$M_{low} = 1111$	$M_{low} = 555$
256	INC	48.35	71.82	64.07	101.09
(16×16)	U-channels	47.71	71.70	63.24	100.83
1024	INC	85.00	112.00	101.98	141.79
(32×32)	U-channels	84.19	109.85	100.96	142.64
512	INC	22.85	44.23	34.69	69.33
$(8 \times 8 \times 8)$	U-channels	22.29	42.47	33.94	66.97
4096	INC	96.29	125.59	111.28	156.19
$(16 \times 16 \times 16)$	U-channels	94.86	122.92	109.59	152.64

with the uniform distribution of message destination and up to 86% with the complement traffic pattern[1]. In general, the results are slightly better for the **INC** mechanism than for the **U-channels** mechanism. However, as the differences between both mechanisms do not exceed 5%, we can conclude that the progressive reduction/recovery provided by **INC** it is not an improvement but adds complexity.

Table 3 shows the same results when we use the **INC** and **Self-tuned** congestion control mechanisms versus **No-Lim**. Deadlock avoidance is used. In this case, the **INC** mechanism obtains improvements in the simulation time up to 150% versus **No-Lim** for a uniform distribution of message destinations, and up to a 22.5% for the complement traffic pattern[1]. These improvements are far from the ones achieved by the **Self-Tuned** mechanism. Moreover, **INC** reaches this better performance without transmitting information through the network. Hence, for deadlock avoidance, **INC** is clearly a better option than **Self-tuned**.

[1] This result can be found in [3]

Table 3. Improvement (%) in the time required to deliver all the injected messages for the *INC* and *Self-Tuned* mechanisms with different topologies, fully adaptive routing algorithm with deadlock avoidance and 3 virtual channels. Uniform traffic pattern.

Nodes	Mechanism	$L_{sat} = 1.2 \times T_s$		$L_{sat} = 1.8 \times T_s$	
		$M_{low} = 1111$	$M_{low} = 555$	$M_{low} = 1111$	$M_{low} = 555$
256	INC	20.27	64.94	47.93	106.69
(16×16)	Self-tuned	18.04	58.16	45.16	82.40
1024	INC	113.43	116.00	109.26	150.15
(32×32)	Self-tuned	109.44	93.26	94.59	84.77
512	INC	0.00	2.33	0.00	24.49
($8 \times 8 \times 8$)	Self-tuned	-2.32	-0.22	-1.85	21.45
4096	INC	41.83	83.66	74.36	114.32
($16 \times 16 \times 16$)	Self-tuned	38.09	78.15	70.51	81.77

5 Conclusions

In this paper, we present a mechanism (**INC**) to avoid network saturation based on message throttling. It can be used with routing algorithms based either on deadlock recovery or avoidance schemes, and works well for different network topologies and network loads.

The mechanism tries to estimate network traffic using only local information available to the node, without sending control information through the network. However, it is able to estimate the status of remote areas by measuring the number of flits transmitted by each virtual channel during a fixed interval of time. If this number is lower than a threshold, the mechanism assumes that there is congestion in that virtual channel. In this situation, a flag is set and message injection restrictions are applied. Two kind of flags are used in order to adjust the injection bandwidth, depending on whether the node is already contributing to the congestion or not yet. In the latter case, applied measures are softer (they are only preventive). Moreover, if congestion is repeatedly detected, injection may be completely forbidden during increasing time intervals. Although both the threshold and the forbidden interval time have to be empirically adjusted, we have found that, as the mechanism works in a progressive way, their adjust is not critical.

The evaluation results show that the mechanism is able to avoid performance degradation for all the analyzed conditions, without introducing any penalty for low and medium network loads, when none congestion control mechanism is required. Moreover, it outperforms recent previous proposals.

References

1. E. Baydal, P. López and J. Duato, "A Simple and Efficient Mechanism to Prevent Saturation in Wormhole Networks", *Int. Parallel & Distributed Processing Symp.*, May 2000.
2. E. Baydal, P. López and J. Duato, "Avoiding Network Congestion with Local Information", *Int. Symp. High Performance Computing*, May 2002.

3. E. Baydal, *Una contribución al control de la congestión en redes de interconexión con conmutación wormhole*, (in Spanish), Ph. D. Thesis, Universidad Politécnica de Valencia, Nov. 2002. http://www.gap.upv.es.
4. W. J. Dally and H. Aoki, "Deadlock-Free Adaptive Routing in Multicomputer Networks Using Virtual Channels", *IEEE Trans. on Parallel and Distributed Systems*, vol. 4, no. 4, pp. 466–475, April 1993.
5. J. Duato, "A new theory of deadlock-free adaptive routing in wormhole networks", *IEEE Trans. on Parallel and Distributed Systems*, vol. 4, no. 12, pp. 1320–1331, Dec. 1993.
6. J. Duato and P. López, "Performance evaluation of adaptive routing algorithms for k-ary n-cubes", *Parallel Computer Routing and Communication*, K. Bolding and L. Snyder (ed.), Springer-Verlag, pp. 45–49, 1994.
7. J. Duato, S. Yalamanchili and L.M. Ni, *Interconnection Networks: An Engineering Approach*, 2nd. ed. Morgan Kauffman, 2002.
8. J. H. Kim, Z. Liu and A. A. Chien, "Compressionless routing: A framework for Adaptive and Fault-Tolerant Routing", *IEEE Trans. on Parallel and Distributed Systems*, Vol. 8, No. 3, 1997.
9. F. T. Leighton, *Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes*, San Mateo, CA, USA, Morgan Kaufmann Publishers, 1992.
10. P. López, J.M. Martínez and J. Duato, "A Very Efficient Distributed Deadlock Detection Mechanism for Wormhole Networks", *Int. Symp. on High Performance Computer Architecture*, Feb. 1998.
11. J.M. Martínez, P. López and J. Duato, "A Cost–Effective Approach to Deadlock Handling in Wormhole Networks", *IEEE Trans. on Paral. and Distrib. Processing*, pp. 719–729, July 2001.
12. T.M. Pinkston and S. Warnakulasuriya, "On Deadlocks in Interconnection Networks", *Int. Symp. on Computer Architecture*, June 1997.
13. A. Smai and L. Thorelli, "Global Reactive Congestion Control in Multicomputer Networks", *Int. Conf. on High Performance Computing*, 1998.
14. M. Thottetodi, A.R. Lebeck, S.S. Mukherjee, "Self-Tuned Congestion Control for Multiprocessor Networks", *Technical Report CS-2000-15*, Duke University, Nov. 2000.
15. M. Thottetodi, A.R. Lebeck, S.S. Mukherjee, "Self-Tuned Congestion Control for Multiprocessor Networks", *Int. Symp. on High Performance Computer Architecture*, Feb. 2001.